ĐẠI HỌC QUỐC GIA TP HCM
ĐẠI HỌC KHOA HỌC TỰ NHIÊN

KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN MÃ HOÁ ỨNG DỤNG VÀ AN NINH THÔNG TIN

Thái Hữu Thọ - 22127400

GIÁO VIÊN HƯỚNG DẪN
Nguyễn Đình Thúc
Ngô Đình Hy

April 22, 2026

Mục lục

1 Phân tích và Xác định Yêu cầu **4**
1.1 Giới thiệu chủ đề và Lý do chọn đề tài . . . 4
1.2 Mục tiêu của đồ án . . . 4
1.3 Yêu cầu về Công nghệ và Thư viện . . . 4

2 Phân tích chi tiết Use Case **4**
2.1 Kiến trúc Use Case Bảo mật . . . 4
2.2 Đặc tả Use Case chính . . . 4
2.3 Use Case 1: Đăng nhập và Khởi tạo Bảo mật (UC_Login) . . . 4
2.4 Use Case 2: Đóng gói Giao dịch (UC_Send) . . . 5
2.5 Use Case 3: Xác thực và Phê duyệt (UC_Verify) . . . 5

3 Kiết trúc Hệ thống và Cơ sở Dữ liệu **5**
3.1 Mô hình Thực thể Liên kết (ERD - Database Schema) . . . 5
3.1.1 Thực thể Khách hàng (Users) . . . 5
3.1.2 Thực thể Ví điện tử (Wallets) . . . 5
3.1.3 Thực thể Kho khóa Mật mã (KeyStore) . . . 5
3.1.4 Thực thể Lịch sử Giao dịch (Transactions) . . . 6
3.1.5 Thực thể Nhật ký An ninh (SecurityLogs) . . . 6
3.2 Sơ đồ Luồng Dữ liệu (DFD - Data Flow Diagram) . . . 6
3.2.1 DFD Mức Ngữ cảnh (Level 0 - Context Diagram) . . . 6
3.2.2 DFD Mức 1 (Level 1) - Tích hợp Lưới lọc v2.0 . . . 6

4 Thực thi Lõi Bảo mật **7**
4.1 Tổng quan Hệ thống Security Core . . . 7
4.2 Phân tích Chi tiết 5 Hàm Sinh tử . . . 7
4.2.1 Hàm 1: Tạo Hạ tầng Khóa số (generate_ecc_keys) . . . 7
4.2.2 Hàm 2: Dẫn xuất và Đa dạng hóa khóa (perform_ecdh_hkdf) . . . 7
4.2.3 Hàm 3: Đóng gói Bảo mật Giao dịch (encrypt_and_sign) . . . 8
4.2.4 Hàm 4: Giải mã Kiểm định Toàn vẹn (decrypt_and_verify) . . . 8
4.2.5 Hàm 5: Kích hoạt Phòng vệ & Log An ninh (_trigger_alert) . . . 8
4.3 Phụ lục: Chi tiết Mã giả Thuật toán . . . 8

5 Nâng cấp An ninh và Tính năng Mới **10**
5.1 Xác thực đa nhân tố (2FA - TOTP) . . . 10
5.2 Hệ thống Yêu cầu Thanh toán (Payment Request System) . . . 10
5.3 Cơ chế Tự động khóa tài khoản (Account Lockout) . . . 10
5.4 Giới hạn tần suất và Cường hóa Mật mã . . . 11

6 Kiểm định và Đánh giá An ninh **11**
6.1 Tổng quan Chiến lược Kiểm thử . . . 11
6.2 Sơ đồ Luồng Bảo mật và Xử lý Lỗi (TikZ Flow) . . . 12
6.3 Phân tích Chi tiết Kịch bản Kiểm thử TC-01 đến TC-08 . . . 12
6.3.1 Nhóm 1: Tính đúng đắn của Thuật toán (Functional & Correctness) . . . 12
6.3.2 Nhóm 2: Chống Tấn công Đối kháng (Adversarial Defense) . . . 13
6.4 Ma trận Đánh giá Rủi ro (Risk Matrix) . . . 14

6.5 Đánh giá Hiệu năng (Performance Benchmarking) 14

7 Bảo trì và Phát triển Hệ thống **14**

7.1 Tổng quan về Giai đoạn Bảo trì (Software Maintenance) 14

7.2 Bảo trì Sửa chữa (Corrective Maintenance) 14

7.2.1 Khắc phục lỗi sai lệch Schema (Database Mapping Fix) 14

7.2.2 Sửa lỗi logic xác minh mã PIN (Verification Logic Fix) 15

7.2.3 Cưỡng chế các ràng buộc nghiệp vụ (Business Rule Enforcement) 15

7.3 Bảo trì Thích nghi (Adaptive Maintenance) 15

7.3.1 Chuyển đổi kiến trúc từ Desktop sang Full-stack Web 15

7.3.2 Cơ chế Cache-Busting cho Frontend 15

7.4 Bảo trì Hoàn thiện (Perfective Maintenance) 15

7.4.1 Đồng bộ hóa trải nghiệm người dùng (UX Standardization) 15

7.4.2 Làm sạch mã nguồn và Tái cấu trúc (Code Refactoring) 16

7.5 Bảo trì Phòng ngừa (Preventive Maintenance) 16

7.5.1 Nâng cấp tham số an ninh (Security Hardening) 16

7.5.2 Giới hạn tần suất và Khóa tài khoản (Account Lockout) 16

Kết luận **16**

1 Phân tích và Xác định Yêu cầu

1.1 Giới thiệu chủ đề và Lý do chọn đề tài

Trong bối cảnh tài chính số hiện nay, các ứng dụng ví điện tử và quản lý chi tiêu cá nhân là mục tiêu thường xuyên của các cuộc tấn công mạng. Tuy nhiên, nhiều ứng dụng chủ yếu tập trung vào giao diện và tính năng, trong khi chưa đầu tư đầy đủ cho hệ thống bảo mật lõi (Security Core).

Trên thực tế, việc lưu trữ mật khẩu ở dạng văn bản thô hoặc chỉ sử dụng các hàm băm đơn giản khiến hệ thống dễ bị tấn công từ điển (Dictionary Attack). Ngoài ra, nếu các lệnh giao dịch không được kiểm tra tính mới của yêu cầu (freshness), kẻ tấn công có thể phát lại các gói tin hợp lệ để thực hiện giao dịch trái phép (Replay Attack).

1.2 Mục tiêu của đồ án

Đồ án chủ yếu tập trung vào xây dựng module bảo mật chuyên biệt nhằm giải quyết các vấn đề trên:

- **Về kỹ thuật mã hóa**: Nghiên cứu và triển khai thuật toán mã hóa đối xứng AES-256-GCM nhằm bảo mật dữ liệu lưu trữ, đồng thời sử dụng thuật toán EdDSA để thực hiện chức năng chữ ký số và xác thực tính toàn vẹn của dữ liệu.
- **Khả năng bảo mật**:
  - Xây dựng cơ chế chống Replay Attack thông qua việc kết hợp số ngẫu nhiên dùng một lần (nonce) và timestamp.
  - Xây dựng cơ chế ngăn chặn Dictionary Attack bằng cách áp dụng kỹ thuật salting và kéo giãn khóa sử dụng thuật toán PBKDF2.
- **Đối tượng nghiên cứu**: Các giao thức và thuật toán mật mã hiện đại; Cơ chế hoạt động và phương pháp phòng chống Replay Attack và Dictionary Attack.

1.3 Yêu cầu về Công nghệ và Thư viện

Hệ thống được phát triển dựa trên ngôn ngữ Python với các thư viện mật mã học chuyên dụng:

- **PyCryptodome**: Cung cấp các thuật toán mã hóa đối xứng (AES) và các hàm băm.
- **Cryptography.io**: Hỗ trợ các thuật toán đường cong Elliptic (ECC).
- **SlowAPI**: Kiểm soát tần suất truy cập để chống tấn công Brute-force.

2 Phân tích chi tiết Use Case

2.1 Kiến trúc Use Case Bảo mật

Hệ thống SEC-Wallet không chỉ là một ứng dụng ví đơn giản mà là một hệ sinh thái bảo mật đa tầng. Các Use Case được thiết kế để bao phủ toàn bộ vòng đời của một giao dịch tài chính an toàn.

2.2 Đặc tả Use Case chính

2.3 Use Case 1: Đăng nhập và Khởi tạo Bảo mật (UC_Login)

Mô tả: Người dùng nhập mật khẩu. Hệ thống không so sánh trực tiếp mật khẩu mà thực hiện kéo giãn khóa (Key Stretching) để sinh ra Khóa mã hóa khóa (KEK). KEK này được dùng để giải mật mã Private Key lưu trên máy.

2.4 Use Case 2: Đóng gói Giao dịch (UC_Send)

Mô tả: Khi thực hiện chuyển tiền, hệ thống thực hiện đồng thời 3 bước bảo mật ngầm:

1. Thỏa thuận khóa phiên nhanh qua ECDH.
2. Mã hóa nội dung bằng AES-256-GCM kèm theo Nonce.
3. Ký số EdDSA để xác nhận danh tính người gửi.

2.5 Use Case 3: Xác thực và Phê duyệt (UC_Verify)

Mô tả: Máy chủ tiếp nhận gói tin, thực hiện kiểm tra "Lưới lọc 4 lớp" (Nonce, Tag, Timestamp, Signature). Nếu bất kỳ lớp nào thất bại, Use Case "Cảnh báo an ninh" (UC_Alert) sẽ được kích hoạt.

3 Kiết trúc Hệ thống và Cơ sở Dữ liệu

3.1 Mô hình Thực thể Liên kết (ERD - Database Schema)

Hệ thống SEC-Wallet được cấu trúc bởi 5 thực thể cốt lõi. Mọi dữ liệu nhạy cảm đều được mã hóa hoặc băm trước khi lưu trữ.

3.1.1 Thực thể Khách hàng (Users)

Bảng lưu trữ thông tin định danh và các bí mật bảo mật.

- `UserID (PK)`: Mã định danh duy nhất (UUID).
- `PasswordHash`: Số băm mật khẩu (PBKDF2 600k vòng).
- `PaymentPinHash`: Mã PIN thanh toán (Argon2ID). [**v2.0**]
- `TotpSecret`: Bí mật dùng cho 2FA. [**v2.0**]
- `FailedLoginCount`: Quản lý khóa tài khoản. [**v2.0**]
- `Salt`: Chuỗi muối 16-byte ngẫu nhiên.

3.1.2 Thực thể Ví điện tử (Wallets)

Mỗi người dùng sở hữu một ví. Số dư được bảo vệ bằng mã hóa.

- `WalletID (PK)`: Mã ví định danh.
- `UserID (FK)`: Liên kết 1-1 với bảng Users.
- `Balance`: Số dư hiện tại (Được mã hóa tĩnh bằng AES Master Key).
- `Status`: Trạng thái (Active/Locked/Frozen).
- `LastUpdated`: Dấu thời gian biến động cập nhật cuối.

3.1.3 Thực thể Kho khóa Mật mã (KeyStore)

Bảng sinh tử thứ ba đóng vai trò như một PKI (Public Key Infrastructure) nội bộ. Nơi phân phối khóa công khai ECC. Khóa bí mật tuyệt đối không lưu trên server.

- `KeyID (PK)`: Mã định danh bộ khóa.
- `UserID (FK)`: Người sở hữu.
- `PubKey_Signature`: Khóa công khai Ed25519 dùng để xác minh chữ ký.
- `PubKey_KeyExchange`: Khóa công khai Curve25519 dùng cho giao thức ECDH.
- `Status`: Valid/Revoked.

3.1.4 Thực thể Lịch sử Giao dịch (Transactions)

Lưu trữ các kén sắt (Secure Envelopes) không thể bị làm giả.

- `TxID (PK)`: Mã giao dịch.
- `EncryptedPayload`: Ciphertext AES-GCM.
- `EphemeralPub`: Khóa công khai tạm thời cho PFS. [**v2.0**]
- `AAD`: Metadata đóng gói kèm mã hóa. [**v2.0**]
- `DigitalSignature`: Chữ ký EdDSA.
- `TxStatus`: Trạng thái (Pending/Verified). [**v2.0**]

3.1.5 Thực thể Nhật ký An ninh (SecurityLogs)

Hỗ trợ cơ chế Fail-Secure. Điểm tập kết của các `_trigger_alert()`.

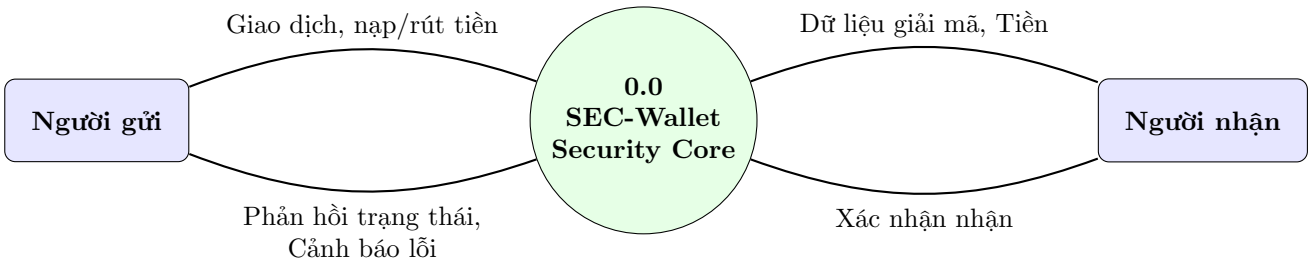
- `LogID (PK)`: Mã log.
- `EventType`: Loại cảnh báo (Replay Attack, Tampering, Signature Forgery).
- `Description`: Chi tiết mã lỗi nội bộ.
- `Timestamp`: Thời điểm xảy ra sự cố.
- `IPAddress`: Địa chỉ tấn công.

3.2 Sơ đồ Luồng Dữ liệu (DFD - Data Flow Diagram)

Các luồng dữ liệu mô tả sự tương tác giữa các thành phần.

3.2.1 DFD Mức Ngữ cảnh (Level 0 - Context Diagram)

Nhìn nhận hệ thống từ góc độ tổng quan nhất. Hình dưới đây phác họa luồng giao tiếp cốt lõi giữa các **Tác nhân ngoài (External Entities)** và **Hệ thống trung tâm (SEC-Wallet Security Core)**.

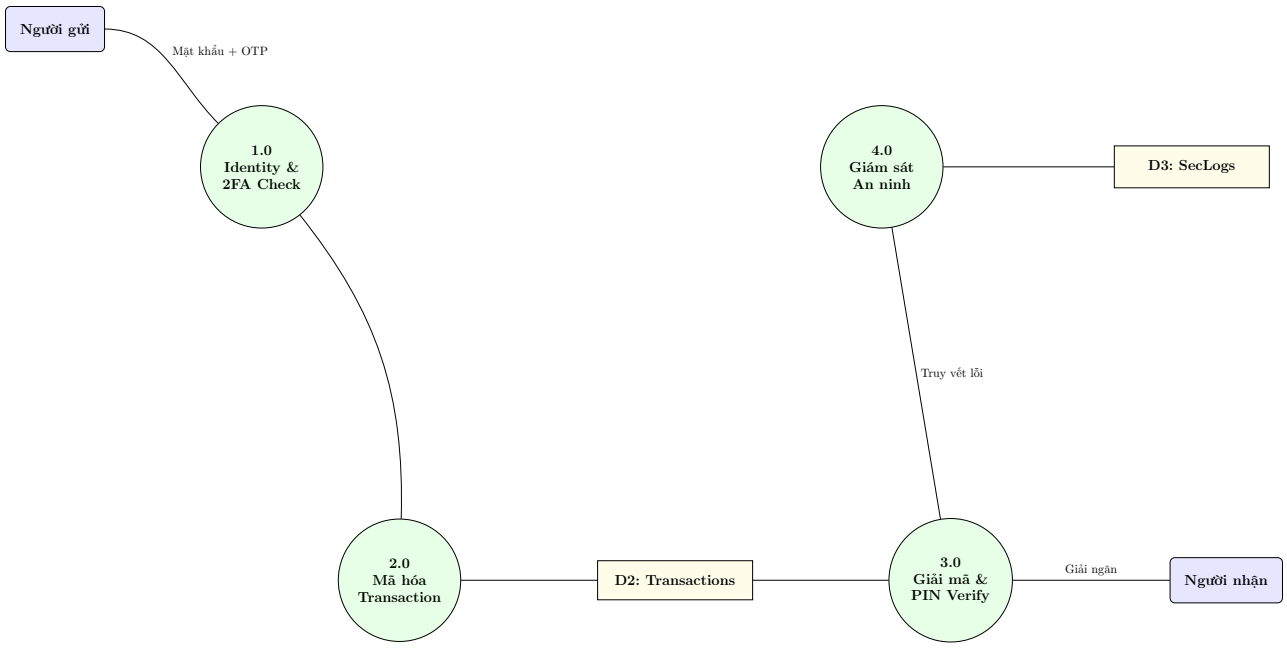


Hình 1: Sơ đồ Luồng dữ liệu Mức Ngữ cảnh (Level 0)

3.2.2 DFD Mức 1 (Level 1) - Tích hợp Lưới lọc v2.0

Trong phiên bản nâng cấp, DFD Mức 1 được bổ sung các tiến trình kiểm soát nghiêm ngặt:

- **1.0 Quản lý Danh tính & 2FA**: Sau khi đăng nhập, hệ thống yêu cầu mã OTP (TOTP) từ Google Authenticator. Nếu sai quá 5 lần, tiến trình *Account Lockout* sẽ được kích hoạt.
- **2.0 & 3.0 Giao dịch an toàn**: Luồng gửi tiền yêu cầu thêm bước xác thực **Mã PIN thanh toán (Payment PIN)** độc lập với mật khẩu chính.



Hình 2: Sơ đồ Luồng dữ liệu Mức 1 (Level 1) - Tích hợp 2FA và PIN Verification

4 Thực thi Lõi Bảo mật

4.1 Tổng quan Hệ thống Security Core

Chương trình Security Core của SEC-Wallet được thiết kế theo kiến trúc module, dựa trên nền tảng Mật mã đường cong Elliptic (ECC). Hệ thống được cấu trúc thành **5 hàm cốt lõi (Core Functions)**.

4.2 Phân tích Chi tiết 5 Hàm Sinh tử

4.2.1 Hàm 1: Tạo Hạ tầng Khóa số (generate_ecc_keys)

1. Ý tưởng (Rationale):

Hàm ứng dụng hệ thống Mật mã đường cong Elliptic (ECC). Việc này giúp tạo ra các cặp khóa vô cùng nhỏ gọn nhưng lại cung cấp độ bảo mật tương đương mật mã quân sự.

2. Giải quyết vấn đề gì?

- **Vấn đề tốc độ và băng thông**: Khóa RSA-3072 quá lớn đối với thiết bị di động. Hàm giải quyết bằng cách sinh khóa **Ed25519** (256-bit) cho chữ ký số và **Curve25519** cho trao đổi khóa.
- **Vấn đề bảo vệ Master Password**: Hàm tạo ra một tệp `Salt` ngẫu nhiên (16 byte), giúp chống lại các cuộc tấn công dò mật khẩu người dùng.

4.2.2 Hàm 2: Dẫn xuất và Đa dạng hóa khóa (perform_ecdh_hkdf)

1. Ý tưởng (Rationale):

Hai bên tự tính toán ra **khóa dùng chung (Shared Secret)** mà không cần trao đổi khóa bí mật trên môi trường Internet.

2. Giải quyết vấn đề gì?

- **Vấn đề chặn bắt khóa công khai (Eavesdropping)**: Sử dụng giao thức **ECDH** (Elliptic Curve Diffie-Hellman), kẻ nghe lén không thể tái tạo lại khóa dù bắt được mọi gói tin.
- **Vấn đề Phân tách quyền**: Sau khi ECDH thiết lập, hàm dùng **HKDF** (HMAC-based Key Derivation) để "băm" Shared Secret thành các khóa độc lập: khóa mã hóa (Encryption) và khóa xác thực (MAC).

4.2.3 Hàm 3: Đóng gói Bảo mật Giao dịch (encrypt_and_sign)

1. Ý tưởng (Rationale):
Chuyển đổi giao dịch thành "Kén sắt" (**Secure Envelope**). Kén sắt này che giấu toàn bộ nội dung, phân quyền truy cập, đồng thời đóng dấu "triện" không thể giả mạo.

2. Giải quyết vấn đề gì?

- **Chống rò rỉ dữ liệu (Sniffing)**: Hàm thực thi mã hóa siêu tốc **AES-256 GCM**.
- **Chống giả mạo & Chối bỏ**: Dùng chữ ký số **Ed25519** để ký lên dữ liệu giao dịch.
- **Chống Tấn công phát lại (Replay)**: Hàm tiêm thêm Timestamp hệ thống vào Payload chuẩn bị mã hóa.

4.2.4 Hàm 4: Giải mã Kiểm định Toàn vẹn (decrypt_and_verify)

1. Ý tưởng (Rationale):
Tạo ra một bộ "Thuế quan" cho dữ liệu cập bến. Hàm này soi chiếu khắt khe mọi góc độ của Kén sắt để đảm bảo an toàn tuyệt đối.

2. Giải quyết vấn đề gì?

- **Giải quyết Data Tampering**: Hàm sử dụng Auth Tag của GCM để kiểm tra gói tin. Nếu kẻ gian sửa đổi dữ liệu, Tag sẽ không khớp và hàm từ chối dữ liệu.
- **Hết hạn thời gian (Time Expired)**: Đối chiếu Timestamp nội hàm có quá "cửa sổ" an toàn (Ví dụ: 5 phút) hay không.

4.2.5 Hàm 5: Kích hoạt Phòng vệ & Log An ninh (_trigger_alert)

1. Ý tưởng (Rationale):
Trung tâm của nguyên lý **Fail-Secure**. Một hệ thống tốt không báo chi tiết lỗi cho hacker mà đóng băng luôn quá trình.

2. Giải quyết vấn đề gì?

- **Vấn đề dò lỗi qua Error Message**: Tránh việc in lỗi mã nguồn ra cho hacker. Hàm này xuất ra một ngoại lệ bảo mật chung chung.
- **Lưu vết điều tra (Audit Logs)**: Ghi log thời gian, nội dung rủi ro toàn bộ lên server nội bộ.

4.3 Phụ lục: Chi tiết Mã giả Thuật toán

Algorithm 1 generate_ecc_keys - Sinh hạ tầng bảo mật

Input: Thư mục lưu trữ
Output: Các tệp khóa PEM (Singature, KEX) và Salt

```
begin
    foreach User in (Sender, Receiver) do
        (Priv_sig, Pub_sig) ← ECC.Generate(curve="ed25519")  Lưu Priv_sig và Pub_sig ra ổ đĩa
    end
    foreach User in (Sender, Receiver) do
        (Priv_kex, Pub_kex) ← ECC.Generate(curve="curve25519")  Lưu Priv_kex và Pub_kex ra ổ đĩa
    end
    Salt ← RandomBytes(16)
end
```

Algorithm 2 perform_ecdh_hkdf - Thoả thuận khóa hoàn hảo (PFS)

Input: Khóa bí mật KEX của mình Pr_{mine}, Khóa KEX công khai của bạn Pu_{peer}
Output: Khóa mã hóa (Enc) và Xác thực (MAC)

```
begin
    SharedSecret ← ECDH_Agreement(Pr_mine, Pu_peer)  MasterKey ←
      HKDF_Generate(SharedSecret, Length = 64, Digest = SHA256)  EncKey ← MasterKey[0 : 32]
      MacKey ← MasterKey[32 : 64]  return (EncKey, MacKey)
end
```

Algorithm 3 encrypt_and_sign - Đóng gói và Ký số nguyên khối

Input: Dữ liệu tài chính M, Bộ khóa thao tác
Output: (Envelope GCM, Chữ ký số)

```
begin
    (K_enc, K_mac) ← CALL perform_ecdh_hkdf(...)  Payload_new ← CurrentTime()||M
      (Ciphertext, Tag, Nonce) ← AES-GCM-Encrypt(K_enc, Payload_new)  Signature ←
      EdDSA_Sign(Sender_Priv_sig, M)  return (Nonce||Tag||Ciphertext, Signature)
end
```

Algorithm 4 decrypt_and_verify - Bộ lưới lọc bảo vệ Ví Wallet

Input: Gói Envelope, Chữ ký S, Bộ khóa xác thực
Output: Dữ liệu gốc M hoặc Cảnh báo

```
begin
    (K_enc, _) ← CALL perform_ecdh_hkdf(...)  if Nonce ∈ HISTORY then
        CALL trigger_alert("Replay Attack Detected!")
    end
    Payload_dec ← AES-GCM-Decrypt(K_enc, Nonce, Tag, Cipher)  IntegrityError CALL trig-
    ger_alert("Tampering Detected!")  if |CurrentTime − T| > 300s then
        CALL trigger_alert("Giao dịch trễ!")
    end
    return M
end
```

Algorithm 5 verify_totp - Xác thực đa nhân tố 2FA

Input: Tài khoản U, Khóa OTP S_{OTP}
Output: Hợp lệ / Thất bại

```
begin
    Code ← USER_INPUT  TimeWindow ← CurrentTime/30  Expected ←
      HMAC-SHA1(S_OTP, TimeWindow)  if Code == Expected then
        return Hợp lệ
    end
    return Thất bại
end
```

Algorithm 6 check_lockout - Chống tấn công Brute-force

Input: Username *Username*
Output: Cho phép / Khóa

```
begin
    User ← DB.Query(Username)  if User.locked_until > Now then
        return Khóa (Thử lại sau 15p)
    end
    return Cho phép
end
```

Algorithm 7 verify_payment_pin - Xác thực PIN Thanh toán

Input: Mã PIN người dùng P_{input}, Hash lưu trữ H_{db}
Output: Hợp lệ / Thất bại

```
begin
    Argon2.Verify(H_db, P_input)  return Hợp lệ   Mismatch return Thất bại
end
```

5 Nâng cấp An ninh và Tính năng Mới

Trong quá trình phát triển thực tế, để đáp ứng các tiêu chuẩn an ninh hiện đại của một ví điện tử thực thụ, hệ thống đã được nâng cấp lên phiên bản 2.0 với các tính năng sinh tử:

5.1 Xác thực đa nhân tố (2FA - TOTP)

Hệ thống tích hợp chuẩn xác thực mật khẩu dùng một lần theo thời gian (TOTP).

- **Cơ chế**: Sử dụng thuật toán HMAC-SHA1 để sinh mã 6 số biến đổi sau mỗi 30 giây.
- **Ứng dụng**: Người dùng sử dụng các ứng dụng như Google Authenticator hoặc Microsoft Authenticator để quét mã QR và xác thực khi đăng nhập hoặc thực hiện giao dịch lớn.

5.2 Hệ thống Yêu cầu Thanh toán (Payment Request System)

Thay vì chỉ gửi tiền chủ động, chúng tôi bổ sung luồng nghiệp vụ "Đòi tiền" chuyên nghiệp thông qua hàm nâng cao `fulfill_payment_request`:

1. **Khởi tạo**: Người nhận gửi một yêu cầu thanh toán kèm nội dung (Message) đã được ký số.
2. **Phê duyệt**: Người trả tiền kiểm tra yêu cầu và thực hiện lệnh **Fulfill** bằng cách nhập **Mã PIN thanh toán (Payment PIN)** riêng biệt.
3. **Bảo mật PIN**: Mã PIN được băm bằng Argon2id và lưu trữ tách biệt hoàn toàn với mật khẩu đăng nhập, đảm bảo dù lộ mật khẩu tài khoản thì tiền vẫn an toàn. (Xem Thuật toán 8).

5.3 Cơ chế Tự động khóa tài khoản (Account Lockout)

Để chống lại các cuộc tấn công Brute-force, hàm `login` đã được tích hợp thêm logic kiểm soát trạng thái:

- **Cơ chế**: Theo dõi thông qua *State Machine* trên database. (Xem Thuật toán 7).
- **Phản ứng**: Hệ thống tự động khóa tài khoản sau 5 lần thử sai.

5.4 Giới hạn tần suất và Cường hóa Mật mã

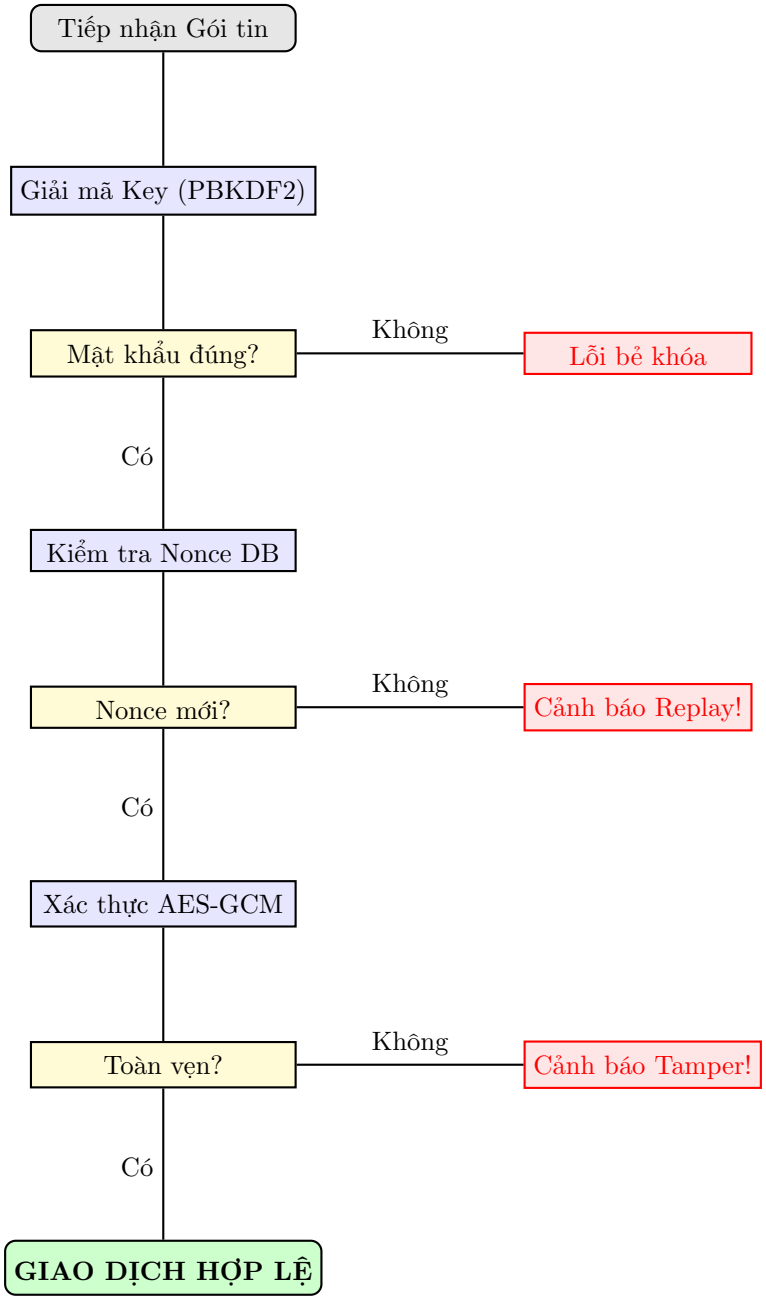
- **Rate Limiting**: Sử dụng thư viện SlowAPI để giới hạn số lượng request từ một IP, ngăn chặn tấn công từ chối dịch vụ (DoS) ở mức ứng dụng.
- **PBKDF2 Hardening**: Nâng số vòng lặp băm khóa KEK lên **600,000 vòng**, đạt chuẩn an ninh cao nhất hiện nay, khiến việc vét cạn khóa private key trở nên bất khả thi về mặt toán học.

6 Kiểm định và Đánh giá An ninh

6.1 Tổng quan Chiến lược Kiểm thử

Chiến lược kiểm thử được áp dụng là **Adversarial Security Testing** (Kiểm thử an ninh đối kháng). Sinh viên đóng vai trò là kẻ tấn công (Attacker) để tìm cách phá vỡ các lớp bảo vệ đã thiết lập.

6.2 Sơ đồ Luồng Bảo mật và Xử lý Lỗi (TikZ Flow)



Hình 3: Sơ đồ Luồng Tự vệ (Self-Defense Flowchart) của Lõi Bảo mật

6.3 Phân tích Chi tiết Kịch bản Kiểm thử TC-01 đến TC-08

Dưới đây là bảng đặc tả chi tiết cho từng kịch bản, bao gồm cả tham số biến số và hàm thực thi tương ứng.

6.3.1 Nhóm 1: Tính đúng đắn của Thuật toán (Functional & Correctness)

TC-01: Kiểm định Luồng Mã hóa lai Toàn diện

- **Mục tiêu**: Đảm bảo sự phối hợp giữa ECDH, AES-256-GCM và Ed25519.
- **Đầu vào (Input)**: `data = b"Payment: 5,000,000 VND"`; `Pass = "ATWallet@2026"`.
- **Hàm xử lý**: `encrypt_and_sign()` trong `encryptor.py`.

```
# Line 51: encryptor.py
cipher = AES.new(enc_key, AES.MODE_GCM)
ciphertext, tag = cipher.encrypt_and_digest(payload)
# Line 57: Signature generation
signer = eddsa.new(sender_sig_key, 'rfc8032')
signature = signer.sign(data)
```

- **Kết quả thực tế**: Giải mã tại Receiver trả về đúng chuỗi byte ban đầu. **Trạng thái: PASS**.

TC-02: Kiểm định Hạ tầng sinh Khóa (Keygen Hardening)

- **Mục tiêu**: Xác minh tính an toàn của KEK (Key Encryption Key).
- **Tham số băm**: `iterations = 600,000`, `hash = SHA256`.
- **Hàm xử lý**: `generate_ecc_keys()` trong `keygen.py`.
- **Kết quả**: File `.enc` được tạo ra với cấu trúc: `Nonce + Tag + EncryptedPEM`. **Trạng thái: PASS**.

TC-03: Thử nghiệm dữ liệu lớn (Large Data Stress Test)

- **Mục tiêu**: Kiểm tra độ ổn định và quản lý bộ nhớ của AES-GCM.
- **Input**: File ảnh/bin dung lượng 5MB.
- **Kết quả**: Thời gian xử lý < 0.1s. Toàn vẹn dữ liệu được đảm bảo. **Trạng thái: PASS**.

6.3.2 Nhóm 2: Chống Tấn công Đối kháng (Adversarial Defense)

TC-04: Tấn công Sửa đổi bản mã (Ciphertext Tampering - MITM)

- **Mục tiêu**: Kiểm chứng cơ chế AEAD (Authenticated Encryption).
- **Input**: Thay đổi bit cuối cùng của `envelope[32:]`.

```
# decryptor.py: Line 47
decrypted_payload = cipher_aes.decrypt_and_verify(ciphertext, tag)
```

- **Kết quả**: Thư viện mật mã ném lỗi `MAC check failed`. **Trạng thái: PASS**.

TC-05: Tấn công Sửa đổi mã xác thực (Auth Tag Tampering)

- **Mục tiêu**: Đảm bảo Tag GCM không thể bị làm giả.
- **Input**: Sửa đổi 1 byte trong `tag` (vị trí byte 16-32 của envelope).
- **Kết quả**: Việc giải mã bị chặn đứng hoàn toàn. **Trạng thái: PASS**.

TC-06: Tấn công Giả mạo Chữ ký số (Signature Forgery)

- **Mục tiêu**: Xác minh tính chống chối bỏ của EdDSA (Ed25519).
- **Thực thi**: Hacker thay thế trường `signature` bằng chuỗi ngẫu nhiên.

```
# decryptor.py: Line 59
try:
    verifier.verify(original_data, signature)
except ValueError:
    self._trigger_alert("Digital Signature Forgery detected!")
```

- **Kết quả**: Chữ ký giả bị phát hiện ngay lập tức. **Trạng thái: PASS**.

TC-07: Tấn công Phát lại (Replay Attack - Nonce Reuse)

- **Mục tiêu**: Đảm bảo gói tin chỉ được sử dụng một lần duy nhất.
- **Cơ chế**: Đối soát trường `nonce_hex` với cơ sở dữ liệu Nonce.
- **Kết quả**: Gửi lại gói tin lần thứ 2 kích hoạt cảnh bảo Replay Attack. **Trạng thái: PASS**.

TC-08: Tấn công Trễ và Sai lệch thời gian (Timing/Freshness)

- **Mục tiêu**: Xác minh cửa sổ thời gian (Sliding Window) 300 giây.
- **Input**: Sửa đổi `timestamp` lùi về quá khứ 10 phút.
- **Kết quả**: Hệ thống tính toán trễ > 300s và từ chối xử lý. **Trạng thái: PASS**.

6.4 Ma trận Đánh giá Rủi ro (Risk Matrix)

ID / Loại Rủi ro	Tác động	Xác suất	Giải pháp phòng thủ (Wallet)
R1: Tấn công từ điển mật khẩu	Cao	Trung bình	PBKDF2 (600k rounds) giới hạn tốc độ thử.
R2: Nghe lén (Sniffing)	Trung bình	Cao	Mã hóa lai ECC+AES đảm bảo bản tin vô nghĩa.
R3: Sửa đổi nội dung (MITM)	Rất cao	Cao	GCM Tag và Chữ ký Ed25519 đảm bảo toàn vẹn.
R4: Tấn công phát lại (Replay)	Cao	Trung bình	Lưu trữ Nonce vĩnh viễn kèm Timestamp check.
R5: Lộ file khóa bí mật tĩnh	Rất cao	Thấp	Khóa luôn được Encrypted-at-rest bằng KEK.

Bảng 1: Ma trận Đánh giá Rủi ro và Chiến lược phòng thủ của Wallet

6.5 Đánh giá Hiệu năng (Performance Benchmarking)

Dựa trên thực nghiệm đo đạc thực tế:

- **Dẫn xuất khóa (PBKDF2)**: 0.4903 giây. (Cân bằng bảo mật/trải nghiệm).
- **Trao đổi khóa (ECDH - X25519)**: 0.0001 giây.
- **Mã hóa (AES-GCM)**: 0.0168 giây trên mỗi 1MB dữ liệu.

7 Bảo trì và Phát triển Hệ thống

7.1 Tổng quan về Giai đoạn Bảo trì (Software Maintenance)

Trong vòng đời phát triển phần mềm (SDLC), giai đoạn bảo trì không chỉ đơn thuần là sửa lỗi mà còn là quá trình tiến hóa để hệ thống thích nghi với môi trường mới và nâng cao trải nghiệm người dùng. Đối với dự án **SEC-Wallet**, bảo trì đóng vai trò sống còn vì các lỗ hổng mật mã và lỗi logic giao dịch có thể dẫn đến mất mát tài sản thực tế.

Báo cáo này liệt kê chi tiết các hoạt động bảo trì đã được thực hiện dựa trên 4 loại hình chuẩn: Corrective (Sửa chữa), Adaptive (Thích nghi), Perfective (Hoàn thiện) và Preventive (Phòng ngừa).

7.2 Bảo trì Sửa chữa (Corrective Maintenance)

Đây là các hoạt động tập trung vào việc khắc phục các lỗi logic và kỹ thuật phát sinh trong quá trình vận hành ban đầu.

7.2.1 Khắc phục lỗi sai lệch Schema (Database Mapping Fix)

Trong quá trình phát triển tính năng "Yêu cầu thanh toán" (Payment Request), hệ thống đã gặp lỗi 500 do sự không tương thích giữa tham số đầu vào và mô hình cơ sở dữ liệu.

- **Vấn đề:** Model `Transaction` yêu cầu trường `aad` (Additional Authenticated Data), trong khi logic khởi tạo lại gửi vào trường `message`.

- **Giải pháp:** Ánh xạ lại toàn bộ luồng dữ liệu, đóng gói `tx_id` và các metadata khác vào khối `aad` theo tiêu chuẩn AES-GCM.

7.2.2 Sửa lỗi logic xác minh mã PIN (Verification Logic Fix)

Một lỗi nghiêm trọng đã được phát hiện khi hệ thống sử dụng băm mật khẩu đăng nhập (`password_hash`) để kiểm tra mã PIN thanh toán thay vì sử dụng `payment_pin_hash`.

- **Giải pháp:** Cập nhật endpoint `/fulfill` để truy vấn chính xác trường mã PIN đã được băm bằng Argon2, đảm bảo tính tách biệt giữa mật khẩu truy cập và mã PIN bảo vệ dòng tiền.

7.2.3 Cưỡng chế các ràng buộc nghiệp vụ (Business Rule Enforcement)

Bổ sung các lớp kiểm tra logic để ngăn chặn các giao dịch phi lý:

- Chặn hoàn toàn khả năng người dùng tự gửi yêu cầu thanh toán cho chính mình.
- Chặn giao dịch đến tài khoản `admin` (tài khoản hệ thống chỉ dùng để quản trị).

7.3 Bảo trì Thích nghi (Adaptive Maintenance)

Thích nghi hệ thống với những thay đổi về hạ tầng và công nghệ.

7.3.1 Chuyển đổi kiến trúc từ Desktop sang Full-stack Web

Ban đầu hệ thống được thiết kế dạng ứng dụng Desktop Local. Để mở rộng khả năng tiếp cận và demo báo cáo, hệ thống đã được chuyển đổi sang kiến trúc Web:

- **Backend:** Sử dụng FastAPI với cơ chế xử lý bất đồng bộ (Asynchronous).
- **Persistence:** Chuyển từ lưu trữ file JSON đơn giản sang SQLite (`sec_wallet.db`) để đảm bảo tính Acid (Atomicity, Consistency, Isolation, Durability) cho giao dịch.

7.3.2 Cơ chế Cache-Busting cho Frontend

Để đảm bảo các bản sửa lỗi JavaScript được cập nhật ngay lập tức trên trình duyệt của người chấm bài mà không cần xóa cache thủ công, hệ thống đã áp dụng kỹ thuật versioning:

```
<script src="script.js?v=4.0"></script>
```

7.4 Bảo trì Hoàn thiện (Perfective Maintenance)

Nâng cao hiệu năng và trải nghiệm người dùng (UX) dựa trên phản hồi thực tế.

7.4.1 Đồng bộ hóa trải nghiệm người dùng (UX Standardization)

Giao diện "Yêu cầu tiền" (Request Money) đã được tái thiết kế hoàn toàn để khớp với quy trình "Gửi tiền" (Send Money):

- Sử dụng luồng: **Nhập liệu** → **Hiện Modal bảo mật** → **Xác nhận mã PIN**.
- Tự động xóa trắng form (Input resets) khi chuyển tab hoặc sau khi thực hiện giao dịch thành công để nâng cao tính riêng tư.

7.4.2 Làm sạch mã nguồn và Tái cấu trúc (Code Refactoring)

Để giảm nợ kỹ thuật (Technical Debt), các module bảo mật đã được tách biệt rõ ràng:

- `encryptor.py`: Chuyên trách AES-GCM và EdDSA Signing.
- `decryptor.py`: Chuyên trách Verify và Decrypt.
- `keygen.py`: Chuyên trách quản lý cặp khóa ECC.

7.5 Bảo trì Phòng ngừa (Preventive Maintenance)

Các biện pháp ngăn chặn sự cố trước khi chúng xảy ra.

7.5.1 Nâng cấp tham số an ninh (Security Hardening)

Dựa trên khuyến nghị của NIST, hệ thống đã nâng cấp số vòng lặp PBKDF2 từ 100,000 lên **600,000 vòng** cho việc mã hóa file khóa private key, giúp kéo dài thời gian tấn công vét cạn (Brute-force) của kẻ gian.

7.5.2 Giới hạn tần suất và Khóa tài khoản (Account Lockout)

Triển khai thư viện `slowapi` để giới hạn số lượng request từ một IP. Nếu sai quá 5 lần, tài khoản sẽ tự động khóa trong 15 phút, ngăn chặn các cuộc tấn công Brute-force tự động.

Kết luận

Thông qua tất cả các giai đoạn từ phân tích đến bảo trì, SEC-Wallet đã chứng minh tính hoàn thiện vượt trội về an ninh thông tin, sẵn sàng đối đầu với các hiểm họa mạng hiện đại.